计算机组成原理 课程总结（考点笔记）

按考点组织的结构化复习笔记

一、计算机系统概述

二、数据的表示与运算

三、存储系统

四、指令系统

五、中央处理器（CPU）

六、总线

七、输入/输出系统

> 使用说明：按「主题 → 考点」组织，覆盖数据表示与运算、存储系统与 Cache、指令系统、CPU 数据通路与流水线、总线、I/O（中断与 DMA），并附常考计算公式。内容依据计算机组成原理通用教材知识体系整理（小林coding 未单独提供计组章节，此份按标准教材框架编写）。公式中 ⌈ ⌉ 表示向上取整。

一、计算机系统概述

1. 冯·诺依曼结构

- **五大部件**：运算器、控制器、存储器、输入设备、输出设备（运算器 + 控制器 = CPU；CPU + 主存 = 主机）。
- **存储程序**：指令和数据以**二进制**形式**同等地位**存放在存储器中，按地址访问；指令由操作码和地址码组成，通常顺序执行。
- 早期以**运算器为中心**，现代计算机以**存储器为中心**。
- 指令和数据在形式上没有区别，CPU 依据**指令周期的不同阶段**区分：取指阶段取出的是指令，执行阶段访问的是数据。

2. 计算机层次结构与软硬件

- 层次：微程序机器 → 传统机器（机器语言）→ 操作系统 → 汇编语言 → 高级语言。
- **编译**：将高级语言程序整体翻译成机器语言目标程序；**解释**：逐条翻译并立即执行，不生成目标程序。
- 软硬件在逻辑功能上**等效**，某些功能既可用硬件也可用软件实现（固件：固化在 ROM 中的软件）。

3. 性能指标

- **字长**：CPU 一次能处理的二进制位数，通常等于通用寄存器位数。
- **主频 f 与 时钟周期 T = 1/f**。
- **CPI**：执行一条指令平均所需时钟周期数。
- **CPU 执行时间 = 指令条数 × CPI × 时钟周期 = 指令条数 × CPI / 主频**。
- **MIPS** = 指令条数 / (执行时间 × 10^6) = **主频 / (CPI × 10^6)**。
- **FLOPS**：每秒浮点运算次数（MFLOPS、GFLOPS、TFLOPS）。
- 吞吐量（单位时间完成的任务量）与响应时间（从提交到完成的时间）。
- 存储容量单位：KB=2^{10}B、MB=2^{20}B、GB=2^{30}B（通信速率中 K=10^3）。

二、数据的表示与运算

4. 进制与编码

- 二、八、十六进制互换：二进制每 3 位对应 1 位八进制，每 4 位对应 1 位十六进制。
- **十进制小数转二进制**：乘 2 取整，顺序排列；有些十进制小数无法用有限位二进制精确表示（如 0.1）。
- **BCD 码**：用 4 位二进制表示 1 位十进制数，常用 8421 码；加法结果大于 9 时需**加 6 修正**。
- **ASCII**：7 位编码，‘0’ = 30H，‘A’ = 41H，‘a’ = 61H。
- **汉字**：区位码 → 国标码（区位码各字节 + 20H）→ 机内码（国标码各字节 + 80H）。

5. 定点数：原码、反码、补码、移码

- **原码**：符号位 + 数值绝对值；0 有 +0、-0 两种表示。
- **反码**：正数同原码；负数符号位不变、数值位取反；0 也有两种表示。
- **补码**：正数同原码；负数 = 反码 + 1（或从右往左第一个 1 之后各位取反）；**0 的表示唯一**；可**把减法转为加法**：[A-B]补 = [A]补 + [-B]补。
- **移码**：在真值上加偏置值 2^{n-1}，即**补码的符号位取反**；0 表示唯一；便于比较大小，**常用于浮点数阶码**。
- **表示范围（n 位整数，含 1 位符号位）**：
  - 原码、反码：**$-(2^{n-1}-1) \sim 2^{n-1}-1$**；
  - 补码、移码：**$-2^{n-1} \sim 2^{n-1}-1$**（多表示一个最小负数，如 8 位补码 1000 0000 表示 -128）。
  - 无符号数：$0 \sim 2^n-1$。
- 求 [-x]补：将 [x]补 **连同符号位一起取反，末位加 1**。
- **符号扩展**：补码扩展时高位填符号位；无符号数扩展时高位补 0。

6. 定点数加减运算与溢出判断

- 补码加减：[A+B]补 = [A]补 + [B]补，[A-B]补 = [A]补 + [-B]补，符号位与数值位一起运算，最高位进位丢弃。
- **溢出只可能发生在同号相加（或异号相减）时**。判断方法：
  1. **单符号位法**：两个操作数符号相同，而结果符号与之不同 → 溢出；
  2. **进位判断法**：符号位进位 Cs 与最高数值位进位 C_1 **异或为 1** → 溢出（$V = Cs \oplus C_1$）；
  3. **双符号位法（变形补码，模 4）**：结果符号位为 **01 → 正溢出，10 → 负溢出**，00 / 11 无溢出。
- **标志位**：
  - **OF**（溢出标志，有符号数）、**SF**（符号标志）、**ZF**（零标志）、**CF**（进位/借位标志，无符号数）；
  - 无符号数减法：CF = 最高位进位 ⊕ 1（即「借位」）；有符号数比较看 OF 与 SF，无符号数比较看 CF。
- **移位运算**：
  - 逻辑移位：空位补 0，用于无符号数；
  - 算术移位（补码）：**左移低位补 0，右移高位补符号位**；左移一位相当于乘 2，右移相当于除 2；
  - 循环移位：移出位补到另一端。

7. 乘法与除法运算

- **原码一位乘**：符号位单独异或，数值位绝对值相乘；从乘数最低位开始，若为 1 则部分积加被乘数，然后部分积与乘数**逻辑右移**一位，共 n 次。
- **补码一位乘（Booth 算法）**：在乘数末尾附加 $y_{(n+1)}=0$，根据 $\mathbf{y_{(n)}y_{(n+1)}}$ 两位判断：
  - **00 或 11**：部分积直接算术右移一位；
  - **01**：部分积加 [x]补 后右移；
  - **10**：部分积加 [-x]补 后右移；
  - 共进行 n+1 次加法、n 次移位（最后一步只加不移）。
- **除法**：
  - **恢复余数法**：余数为负时加回除数恢复，再左移；
  - **不恢复余数法（加减交替法）**：余数为正 → 商 1，左移后**减**除数；余数为负 → 商 0，左移后**加**除数；
  - 定点小数除法要求 |被除数| < |除数|，否则商溢出。

8. 浮点数表示与 IEEE 754

- 一般形式：**$N = (-1)^s \times M \times R^E$**，M 为尾数（决定精度），E 为阶码（决定范围）。

- **规格化**：尾数最高有效位为 1（原码），使精度最高；左规（尾数左移、阶码减 1）、右规（尾数右移、阶码加 1）。
- **IEEE 754**：
  - **单精度 float（32 位）**：1 位符号 + **8 位阶码（偏置 127）** + 23 位尾数；
  - **双精度 double（64 位）**：1 位符号 + **11 位阶码（偏置 1023）** + 52 位尾数；
  - 规格化数尾数**隐含最高位 1**，真值 = $(-1)^s \times$ **1.M** $\times 2^{(E-127)}$。
- **特殊值**（以单精度为例）：
  - 阶码全 0、尾数全 0：**±0**；
  - 阶码全 0、尾数非 0：**非规格化数**，真值 = $(-1)^s \times 0.M \times 2^{(-126)}$；
  - 阶码全 1、尾数全 0：**±∞**；
  - 阶码全 1、尾数非 0：**NaN**。
  - 规格化数阶码范围 1~254，对应指数 -126~+127。
- 例：$-0.75 = -1.1_2 \times 2^{-1}$ → S=1，E = -1+127 = 126 = 0111 1110，M = 100…0 → **BF400000H**。

9. 浮点数加减运算步骤

1. **对阶**：**小阶向大阶看齐**，阶码小的数尾数右移，阶码增加（右移可能损失精度，但避免了高位丢失）；
2. **尾数加减**；
3. **规格化**：结果不规格化时左规或右规；
4. **舍入**：0 舍 1 入、恒置 1，IEEE 754 默认**就近舍入到偶数**，另有朝 0、朝 +∞、朝 -∞ 舍入；
5. **溢出判断**：**阶码上溢**才是浮点数溢出（按机器处理为异常/∞）；阶码下溢按机器零处理；尾数溢出可通过右规解决。

10. 数据的存储与类型转换

- **大端（Big-Endian）**：高位字节存放在低地址（网络字节序）；**小端（Little-Endian）**：低位字节存放在低地址（x86）。例：0x12345678 在小端机器中从低地址起依次为 78 56 34 12。
- **边界对齐**：数据按其长度的整数倍地址存放，以减少访存次数（结构体中会出现填充字节）。
- C 语言类型转换：
  - 同长度有符号 ↔ 无符号：**位模式不变**，解释方式改变（如 short -1 转 unsigned short 为 65535）；
  - 长 → 短：**截断高位**；短 → 长：有符号数符号扩展，无符号数零扩展；
  - int → float：可能**损失精度**（float 只有 24 位有效位）；int → double 不损失；
  - float/double → int：小数部分截断，可能溢出。

11. 运算器与加法器

- **ALU**：完成算术与逻辑运算的组合逻辑电路，输出结果与标志位。
- **一位全加器**：和 $S_i = A_i \oplus B_i \oplus C_i$，进位 $C_{i+1} = A_iB_i + (A_i \oplus B_i)C_i$。
- **串行进位（行波进位）加法器**：进位逐级传递，延迟随位数线性增长。
- **先行进位（并行进位）加法器 CLA**：定义**进位产生函数 $G_i = A_iB_i$**、**进位传递函数 $P_i = A_i \oplus B_i$（或 $A_i + B_i$）**，则 $C_{i+1} = G_i + P_iC_i$，可展开成只依赖 C_0 的表达式，**各位进位同时产生**；位数多时采用组内并行、组间串行或多级先行进位。
- 运算器组成：ALU、累加器 ACC、乘商寄存器 MQ、通用寄存器组、程序状态字寄存器 PSW、移位器等。

12. 校验码

- **奇偶校验码**：增加 1 位校验位使 1 的个数为奇数（奇校验）或偶数（偶校验）；**只能检测奇数位错误，不能纠错**。
- **海明码**：
  - 校验位数 k 满足 **$2^k \geqslant n + k + 1$**（n 为信息位数），如 n=8 需 k=4；
  - 校验位放在 2^0、2^1、$2^2\cdots$ 位置，每个校验位负责检验位置编号二进制中对应位为 1 的那些位；
  - 基本海明码可**纠正 1 位错误**；增加一位总校验位后可**检测 2 位错误、纠正 1 位错误**。
- **CRC 循环冗余校验**：用生成多项式（r+1 位）对信息位左移 r 位后做**模 2 除法**，余数（r 位）作为校验位附在后面；接收端用同一多项式除，余数为 0 则无错。检错能力强，广泛用于网络和存储。

三、存储系统

13. 存储器层次结构与局部性原理

- 层次：**寄存器 → Cache → 主存 → 辅存（磁盘/SSD）**，速度递减、容量递增、单位成本递减。
- **Cache-主存层次**：解决 CPU 与主存**速度不匹配**，由**硬件**自动完成，对程序员透明。
- **主存-辅存层次**：解决主存**容量不足**，由**硬件 + 操作系统**实现虚拟存储器。
- **局部性原理**：
  - **时间局部性**：最近访问的数据很可能再次被访问（循环）；
  - **空间局部性**：访问某地址后，其附近地址很可能被访问（数组顺序访问）。
  - 例：按行遍历二维数组（C 语言行优先存储）比按列遍历的空间局部性好。

14. 存储器分类与性能指标

- 按存取方式：**随机存取 RAM**、**只读 ROM**、**顺序存取（磁带）**、**直接存取（磁盘）**；相联存储器（按内容访问，用于 TLB、Cache 标记比较）。
- 按信息可保存性：**易失性**（RAM）与**非易失性**（ROM、Flash、磁盘）；破坏性读出（DRAM 读后需再生）。
- 性能指标：存储容量 = 存储字数 × 字长；**存取时间**（启动一次操作到完成的时间）；**存取周期**（连续两次独立访问的最小间隔，通常大于存取时间）；**主存带宽** = 每秒传送的信息量。

15. SRAM 与 DRAM、ROM

对比	SRAM	DRAM
存储原理	双稳态触发器	电容存储电荷
是否刷新	不需要	**需要定期刷新**（通常 2ms 内）
速度	快	慢
集成度/容量	低	高
成本/功耗	高	低
地址线	行列地址同时送	**行、列地址分时复用**（地址线减半）
用途	**Cache**	**主存**

- **DRAM 刷新方式**：
  - **集中刷新**：在刷新周期内集中一段时间刷新所有行，期间不能访存（有**死时间**）；
  - **分散刷新**：每个存取周期后刷新一行，无死区但存取周期变长；
  - **异步刷新**：将刷新周期均分给每一行，例如 2ms / 行数，死时间分散，最常用。
  - 刷新以**行**为单位，由存储器内部完成，不需要 CPU 参与。
- **ROM**：MROM（掩模）、PROM（一次可编程）、EPROM（紫外线擦除）、EEPROM（电擦除）、**Flash 闪存**（可快速块擦除，U 盘、SSD 的基础）。

16. 主存容量扩展与地址分配

- **位扩展**：增加字长，各芯片地址线、片选、读写控制**并联**，数据线分别引出。例：用 8K×1 位芯片组成 8K×8 位需 8 片。
- **字扩展**：增加存储字数，地址线、数据线并联，**高位地址经译码产生片选信号**。例：用 8K×8 位组成 32K×8 位需 4 片，2 根高位地址译码。
- **字位同时扩展**：所需芯片数 = (目标容量 / 芯片容量)，即 (M/m) × (N/n)。
- **片选方式**：**线选法**（高位地址每根直接接一个片选，简单但地址空间不连续、浪费）与**译码片选法**（3-8 译码器等，地址连续）。
- 常考：根据地址范围判断芯片数量与片选逻辑；ROM 与 RAM 组合时分别计算区域。

17. 提高主存速度：双端口 RAM 与多模块存储器

- **双端口 RAM**：两套独立的地址、数据、控制线，可同时访问；同时访问**同一单元**时发生冲突，需置忙信号暂停一方。
- **单体多字存储器**：一次读出多个字，要求指令和数据连续存放。
- **多体并行存储器**：
  - **高位交叉编址（顺序方式）**：高位地址选择体，同一体内地址连续，连续访问仍在同一体，**不能提高连续访问速度**，主要用于扩容；
  - **低位交叉编址（交叉方式）**：**低位地址选择体**，连续地址分布在相邻的不同体中，可**流水线方式并行访问**。
- 低位交叉公式：m 个模块，存取周期 T，总线传送周期 r，满足 **T = m · r** 时流水不冲突；**连续读取 m 个字的时间 = T + (m-1) · r**。若连续访问的地址落在同一模块（如间隔为 m 的倍数），则发生访存冲突。

18. Cache 基本原理与性能计算

- 主存与 Cache 都划分为大小相同的**块（行）**，以块为单位交换数据。
- **命中率 h** = 命中次数 / 总访问次数；平均访问时间（tc 为 Cache 访问时间，tm 为主存访问时间）：
  - **CPU 同时访问 Cache 和主存**：t = h · tc + (1-h) · tm；
  - **先访问 Cache，未命中再访问主存**：t = h · tc + (1-h) · (tc + tm)。
- **访问效率** e = tc / t。
- 失效类型：**强制性失效**（首次访问）、**容量失效**、**冲突失效**（映射位置冲突）。
- 增大块大小：开始能利用空间局部性提升命中率，块过大则行数减少、失效损失增大。

19. Cache 映射方式

方式	映射规则	主存地址划分	特点
直接映射	Cache 行号 = 主存块号 mod Cache 行数	**标记 | 行号 | 块内地址**	只需 1 个比较器，速度快；冲突多，命中率低
全相联	主存块可放入任意 Cache 行	**标记 | 块内地址**	命中率高；需要与所有行比较（比较器多、成本高）
组相联（r 路）	组号 = 主存块号 mod 组数，组内任意行	**标记 | 组号 | 块内地址**	折中，比较器个数 = 路数 r

- 计算步骤：块内地址位数 = $\log_2$(块大小)；行号/组号位数 = $\log_2$(行数或组数)；**标记位数 = 主存地址位数 - 行号(组号)位数 - 块内地址位数**。
- **Cache 总容量（含控制位）** = 行数 × (数据位 + **标记位 + 有效位 1 位 + 脏位**（写回法 1 位）+ **替换控制位**（LRU 需 $\log_2 r$ 位））。

20. Cache 替换算法

- 仅在全相联和组相联中需要（直接映射位置固定无需选择）。
- **随机算法 RAND**：实现简单，没有利用局部性。
- **先进先出 FIFO**：替换最早调入的行，未利用局部性，可能出现 **Belady 异常**（增加容量反而命中率下降）。
- **近期最少使用 LRU**：替换最长时间未被访问的行，**利用时间局部性**，命中率高；每行设置计数器（r 路组相联需 $\log_2 r$ 位）：命中时该行清 0，比其小的计数器加 1；未命中时替换计数值最大的行。
- **最不经常使用 LFU**：替换访问次数最少的行。
- 抖动：频繁访问的块被反复换入换出，导致命中率极低。

21. Cache 写策略

- **写命中时**：
  - **全写法（写直达，Write-Through）**：同时写 Cache 和主存，一致性好，但写操作慢 → 增加**写缓冲（Write Buffer）**；
  - **写回法（Write-Back）**：只写 Cache，并设置**脏位（修改位）**，该行被替换时才写回主存，减少访存次数，但存在不一致窗口。
- **写不命中时**：
  - **写分配法**：先把主存块调入 Cache 再写，利用空间局部性，**通常与写回法搭配**；
  - **非写分配法**：只写主存，不调入 Cache，**通常与全写法搭配**。
- 多级 Cache：L1 常采用全写法 + 非写分配（与 L2 之间），L2 采用写回法 + 写分配；L1 通常分为**指令 Cache 与数据 Cache**（哈佛结构，避免结构冒险）。

22. 虚拟存储器与地址转换

- 虚拟存储器：程序使用**虚拟地址（逻辑地址）**，由硬件（MMU）+ 操作系统映射到主存物理地址，使程序可用空间远大于主存。
- **页式**：虚拟空间与主存划分为等长的页（页框），**虚拟地址 = 虚页号 + 页内偏移**；**页表**记录虚页号 → 物理页框号，以及有效位、脏位、引用位、访问权限。页大小固定，无外部碎片，有内部碎片。
- **段式**：按程序逻辑模块分段，段长可变，便于共享与保护，但有外部碎片。
- **段页式**：先分段再分页，地址转换需查段表和页表。
- **TLB（快表）**：存放最近使用页表项的**相联存储器**（Cache 化的页表），避免每次访存都先访问主存中的页表。
- **完整访存过程**：虚拟地址 → **查 TLB**（命中直接得到物理地址）→ 未命中**查主存中的页表**（命中则更新 TLB）→ 页表也无效则**缺页中断**，由操作系统从磁盘调入 → 得到物理地址后**访问 Cache**，未命中再访问主存。

- **TLB、页表、Cache 命中组合**：
  - TLB 命中 ⇒ 页表一定命中（页一定在主存）；
  - **页表未命中（缺页）⇒ TLB 和 Cache 都不可能命中**；
  - 可能的组合：TLB 命中/页表命中/Cache 命中或未命中；TLB 未命中/页表命中/Cache 命中或未命中；三者全未命中。
- 虚拟存储器采用**全相联映射 + 写回法**（磁盘访问代价太高），替换常用 LRU 近似算法；缺页由软件（操作系统）处理，Cache 缺失由硬件处理。

23. 外存：磁盘与固态硬盘

- 磁盘结构：盘面、磁道、扇区、柱面；地址 = 驱动器号 + 柱面（磁道）号 + 盘面号 + 扇区号。
- **平均存取时间 = 平均寻道时间 + 平均旋转延迟 + 传输时间**；**平均旋转延迟 = 旋转半周的时间 = 1 / (2 × 转速)**。
- **数据传输率** = 每道容量 × 每秒转数。
- **RAID**：RAID0（条带化，无冗余，速度快）、RAID1（镜像）、RAID5（分布式奇偶校验，允许坏 1 块盘）、RAID10 等，提升可靠性和性能。
- **SSD**：基于 Flash，无机械部件，随机读写快；以页为单位读写、以块为单位擦除，**写前需擦除**，擦写次数有限 → 需要**磨损均衡**和 FTL（闪存转换层）。

四、指令系统

24. 指令格式与扩展操作码

- 指令 = **操作码 + 地址码**。按地址码个数分为：零地址（如堆栈运算、空操作）、一地址（隐含 ACC）、二地址、三地址指令。
- **定长操作码**：所有指令操作码长度相同，译码简单。
- **扩展操作码**：指令总长固定，**地址码少的指令使用更长的操作码**；要求短操作码不能是长操作码的前缀。
  - 常考计算：如 16 位指令、地址码 4 位，三地址指令用 15 条时，剩余 1 个编码可扩展出 16 条二地址指令，依此类推。
- 指令字长与机器字长关系：半字长、单字长、双字长指令。

25. 寻址方式

寻址方式	有效地址 EA	访存次数（取操作数）	说明
立即寻址	操作数就在指令中	0	速度最快，范围受字段长度限制
直接寻址	EA = A	1	寻址范围受 A 的位数限制
间接寻址	EA = (A)	2（一次间址）	扩大寻址范围，速度慢
寄存器寻址	操作数在寄存器 R_i	0	速度快，指令短
寄存器间接寻址	EA = (R_i)	1	比间接寻址少一次访存
相对寻址	EA = **(PC) + A**	1	用于转移指令，便于程序浮动；注意 PC 通常已指向下一条指令
基址寻址	EA = **(BR) + A**	1	面向**操作系统**，BR 内容由 OS 管理，用于程序重定位，扩大寻址空间
变址寻址	EA = **(IX) + A**	1	面向**用户**，IX 可变、A 不变，适合**数组和循环**
堆栈寻址	由栈顶指针 SP 隐含给出	硬件堆栈 0 / 软件堆栈 1	用于零地址指令、子程序调用与中断

- **指令寻址**：顺序寻址（PC 自增）与跳跃寻址（转移指令修改 PC）；按字节编址时，指令长 n 字节则 PC 增加 n。

26. CISC 与 RISC

对比	CISC（如 x86）	RISC（如 ARM、RISC-V、MIPS）
指令数量	多、复杂	少、简单常用
指令长度	不固定	**固定长度**、格式少
寻址方式	多	少
访存指令	多数指令可访存	**只有 Load/Store 访存**
寄存器	较少	**通用寄存器多**
控制器	多为**微程序控制**	多为**硬布线控制**
流水线	较难实现	易于实现，大多数指令单周期完成
编译	编译相对简单	依赖编译器优化

27. 程序的机器级表示（x86 常考点）

- 常用指令：`mov`（传送）、`lea`（取有效地址，常用于计算）、`add/sub/imul/idiv`、`and/or/xor/not`、`shl/sar/shr`（移位）、`cmp`（相减只设标志）、`test`（相与只设标志）、`jmp` 与条件转移（`je/jne/jg/jl/ja/jb`）、`push/pop`、`call/ret`。
- 有符号比较转移用 `jg/jl`（看 SF、OF），无符号比较用 `ja/jb`（看 CF、ZF）。

- **过程调用栈帧**：`call` 将返回地址压栈并跳转；被调用者 `push ebp; mov ebp, esp` 建立栈帧，局部变量通过 `ebp - 偏移` 访问，参数通过 `ebp + 偏移` 访问（IA-32）；`leave; ret` 恢复并返回。栈从高地址向低地址增长。
- 选择、循环语句对应「比较 + 条件转移」指令序列。

五、中央处理器（CPU）

28. CPU 的功能与组成

- 功能：**指令控制**（程序顺序）、**操作控制**（产生控制信号）、**时间控制**（时序）、**数据加工**（运算）、**中断处理**。
- **运算器**：ALU、累加器 ACC、乘商寄存器 MQ、通用寄存器、暂存寄存器、程序状态字寄存器 **PSW**（含 OF、SF、ZF、CF 及中断允许等标志）、移位器。
- **控制器**：程序计数器 **PC**、指令寄存器 **IR**、指令译码器 ID、存储器地址寄存器 **MAR**、存储器数据寄存器 **MDR**、时序系统、微操作信号发生器。
- **用户可见寄存器**：通用寄存器、PSW、**PC**（可通过转移指令改变）；**用户不可见**：IR、MAR、MDR、暂存寄存器。
- MAR 位数与地址线数相同，MDR 位数与存储字长相同。

29. 指令周期与数据流

- **指令周期**：取出并执行一条指令的全部时间，由若干**机器周期（CPU 周期）**组成，机器周期又由若干**时钟周期**组成。
- 四个阶段（机器周期），用标志触发器 FE、IND、EX、INT 区分：
  - **取指周期**：PC → MAR → 地址总线；主存读 → MDR → IR；PC + 1 → PC；
  - **间址周期**：取出操作数有效地址；
  - **执行周期**：完成指令功能；
  - **中断周期**：保存断点（PC 压栈）、关中断、将中断服务程序入口地址送 PC。
- 取指微操作序列（单总线）：`(PC) → MAR`；`M(MAR) → MDR`，`(PC) + 1 → PC`；`(MDR) → IR`。
- 取指周期和执行周期是每条指令都有的；间址周期和中断周期视情况而定。

30. 数据通路

- **数据通路**：数据在功能部件之间传送的路径，包括 ALU、寄存器、总线、多路选择器等。
- 结构：
  - **CPU 内部单总线**：所有寄存器连接在同一总线上，结构简单，同一时刻只能传送一个数据，ALU 需暂存器 Y、Z 辅助；
  - **多总线**：多个数据可同时传送，效率高；
  - **专用数据通路**：部件之间用专用通路连接，性能好但硬件成本高。

- 常考：根据数据通路图写出某条指令执行阶段的微操作序列与控制信号（如 `ADD R1,(R2)`：取 R2 内容作地址读存储器，与 R1 相加结果写回 R1）。

31. 控制器：硬布线与微程序

- **硬布线控制器**：由组合逻辑电路根据指令操作码、时序信号、状态标志直接产生控制信号；**速度快**，但设计复杂、不易修改扩展，**适合 RISC**。
- **微程序控制器**：将每条机器指令的执行编写为一段**微程序**，存放在**控制存储器 CM（ROM）**中；每条**微指令**包含一组微命令（控制信号）。
  - 机器指令 ↔ 微程序（一对一）；微程序 ↔ 微指令（一对多）；**取指微程序是所有指令公用的**；
  - 微指令格式：**水平型**（一次可定义多个并行微命令，微程序短、速度快、字长长）与**垂直型**（类似机器指令，一条微指令完成一两个操作，字长短、微程序长、速度慢）；
  - 控制字段编码：直接编码（每位一个微命令，速度快、字长长）、**字段直接编码**（**互斥**的微命令分在同一字段编码，每段需留一个「无操作」编码）、字段间接编码；
  - 后继微地址形成：**断定方式**（下址字段）、计数器方式（μPC + 1）、根据操作码由硬件产生入口地址；
  - 特点：设计规整、易修改扩展，但速度比硬布线慢，**适合 CISC**。
- 微命令的**相容性**：能同时出现的为相容，不能同时出现的为互斥。

32. 指令流水线基本概念与性能指标

- 思想：把指令执行分为多个阶段，各阶段由不同部件**重叠并行**处理不同指令。
- 经典五段：**IF（取指）→ ID（译码/读寄存器）→ EX（执行/计算地址）→ MEM（访存）→ WB（写回）**；流水段之间设置流水寄存器，时钟周期取**最长段的时间**（或加上寄存器延迟）。
- 理想情况下 k 段流水线完成 n 条指令需 **(k + n - 1)** 个时钟周期 Δt。
- 性能指标：
  - **吞吐率 TP = n / ((k + n - 1) · Δt)**，n → ∞ 时最大吞吐率为 1/Δt；
  - **加速比 S = 不用流水的时间 / 用流水的时间 = n · k · Δt / ((k + n - 1) · Δt)**，n → ∞ 时趋近 k；
  - **效率 E** = 各段实际使用的时空面积 / 总时空面积 = n · k · Δt / (k · (k + n - 1) · Δt)。
- 适合流水线的指令集特点：指令长度一致、格式规整、Load/Store 型、数据和指令边界对齐。

33. 流水线冒险（相关）与处理

- **结构冒险（资源冲突）**：多条指令同时争用同一硬件资源（如同时访问存储器取指和取数）。解决：**指令 Cache 与数据 Cache 分离**（哈佛结构）、资源重复设置、暂停流水线。
- **数据冒险（数据相关）**：

  - **RAW（写后读）**：后一条指令要读前一条指令尚未写回的结果，**五段顺序流水线中唯一会发生的数据冒险**；
  - WAR（读后写）、WAW（写后写）：在乱序执行、多发射流水线中可能发生；
  - 解决：
    - **暂停/插入气泡（stall/nop）**；
    - **数据旁路（转发 forwarding）**：将 EX/MEM 阶段的结果直接送到下一条指令的 ALU 输入；
    - **Load-Use 冒险**：Load 指令在 MEM 阶段末才得到数据，即使有转发**仍需暂停 1 个周期**；
    - **编译器指令重排**，在相关指令间插入不相关指令。
- **控制冒险（控制相关）**：转移指令改变 PC，流水线中已取入的后续指令可能作废。解决：
  - **分支预测**：静态预测（总预测不跳转/跳转）、**动态预测**（1 位/2 位饱和计数器、分支目标缓冲 BTB）；
  - **延迟分支**：把与分支无关的指令放在分支后的延迟槽中；
  - **尽早判断转移**（在 ID 段计算转移地址和条件）；
  - 预取两个方向的指令。

34. 高级流水线与多处理器

- **超标量**：每个时钟周期**同时发射多条指令**，需多个功能部件，配合动态调度与乱序执行。
- **超流水线**：把流水段划分得更细，提高主频（每个周期内多个流水段处理）。
- **超长指令字 VLIW**：由**编译器**把多条可并行的操作打包成一条长指令。
- **Flynn 分类**：SISD（单处理器）、**SIMD**（向量处理器、GPU 的数据级并行）、MISD（实际很少）、**MIMD**（多处理器、多核、多计算机）。
- **硬件多线程**：细粒度（每周期切换线程）、粗粒度（遇到长停顿切换）、**同时多线程 SMT**（同一周期发射不同线程的指令，如 Intel 超线程）。
- 多核共享主存需要**Cache 一致性协议**（如 MESI）。

六、总线

35. 总线基本概念与分类

- **总线**：连接多个部件的**共享**信息传输线路，分时共享，同一时刻只允许一个部件发送，但可多个部件接收。
- 按功能层次：
  - **片内总线**：CPU 芯片内部；

    - **系统总线**：连接 CPU、主存、I/O 接口，分为**数据总线**（双向，位数与机器字长/存储字长相关）、**地址总线**（单向，位数决定可寻址空间）、**控制总线**（各控制信号线，单根方向固定）；
    - **I/O 总线**、**通信总线**（计算机系统之间）。
- 按时序：同步总线、异步总线；按数据传输格式：串行总线、并行总线。
- 结构：单总线（简单、冲突多）、**双总线**（主存总线 + I/O 总线）、**三总线**（主存总线、I/O 总线、DMA 总线）。

36. 总线性能指标

- **总线宽度**：数据线根数（位数）。
- **总线工作频率**与**总线时钟频率**：每秒传送次数与时钟信号频率。
- **总线带宽（数据传输率）= 总线宽度（字节）× 总线工作频率**；若一次传输需多个时钟周期，则带宽 = 宽度 × 时钟频率 / 每次传输所需时钟周期数。例：32 位总线、时钟 100MHz、每 2 个时钟周期传一次 → 4B × 100M / 2 = 200MB/s。
- **总线复用**：同一组信号线分时传送地址和数据，减少线数。
- **信号线数**：地址线 + 数据线 + 控制线之和。

37. 总线仲裁

- **集中式仲裁**：
    - **链式查询**：总线忙 BS、总线请求 BR、总线允许 BG 三根控制线，BG 信号**串行**逐个传递，离仲裁器越近优先级越高；结构简单，但**对电路故障敏感**，优先级固定，远端设备可能「饿死」；
    - **计数器定时查询**：BS、BR 加上 **$\lceil \log_2 n \rceil$ 根设备地址线**，计数器可从 0 或上次中止处开始，优先级灵活；
    - **独立请求**：每个设备有独立的 BR 和 BG，共 **2n 根**（另加 BS），**响应快、优先级灵活**，但线多、控制复杂。
- **分布式仲裁**：无中央仲裁器，各设备有自己的仲裁号与仲裁器，通过比较仲裁号决定胜者。

38. 总线操作与定时

- 总线周期四阶段：**申请分配**（仲裁）→ **寻址**（主设备发出从设备地址和命令）→ **传输** → **结束**（释放总线）。
- **同步定时**：统一时钟控制，传输速度快、控制简单，但必须按最慢设备设计，适合速度相近的部件、短距离。
- **异步定时**：采用**应答（握手）**方式，无公共时钟，适合速度差异大的设备：
    - **不互锁**：请求、回答信号各自一段时间后自动撤销；
    - **半互锁**：请求信号要等到回答信号到来后才撤销，回答信号自动撤销；

    - **全互锁**：请求与回答信号**相互确认**后才撤销，可靠性最高、速度较慢。
- **半同步**：同步基础上增加「等待（WAIT）」信号，兼顾两者。
- **分离式通信**：一个总线周期分为两个子周期，从设备准备数据期间**释放总线**供其他设备使用，提高总线利用率。
- **突发（猝发）传送**：一次寻址后连续传送多个数据。

39. 常见总线标准

- **PCI**：并行、同步，32/64 位，支持即插即用。
- **PCIe**：**串行、点对点、全双工**，按通道（lane，x1/x4/x16）扩展带宽，取代 PCI/AGP。
- **USB**：串行、支持热插拔和即插即用，可级联多设备。
- **SATA**：串行硬盘接口；**NVMe**：基于 PCIe 的 SSD 协议，延迟低。

七、输入/输出系统

40. I/O 接口与端口编址

- **I/O 接口功能**：地址译码与设备选择、主机与设备之间的**数据缓冲**（速度匹配）、**信号格式转换**（串并转换、电平转换）、传送控制命令与状态信息、中断请求等。
- **端口**：接口中可被 CPU 访问的寄存器，包括**数据端口、状态端口、控制端口**。
- **端口编址方式**：
    - **统一编址（存储器映射 I/O）**：端口与主存共用地址空间，用访存指令访问 I/O，指令灵活，但占用主存地址空间、地址译码复杂；
    - **独立编址（I/O 映射）**：端口有独立地址空间，用专门的 I/O 指令（如 x86 的 `in/out`）访问，程序清晰，但指令种类少。

41. I/O 控制方式对比

方式	原理	CPU 参与程度	特点
程序查询	CPU 循环查询设备状态，就绪后传送	CPU 一直等待（**忙等**）	简单，CPU 利用率低，CPU 与外设**串行**工作
程序中断	设备就绪后发中断请求，CPU 执行中断服务程序传送数据	每传一个数据中断一次	CPU 与外设**并行**；适合低速设备；频繁中断开销大
DMA	由 **DMA 控制器**直接在主存与外设间成块传送	仅在传送**开始预处理和结束后处理**时参与	适合**高速、成批**数据传送（磁盘、网卡）
通道 / I/O 处理器	专用处理器执行通道程序管理 I/O	更少	大型机中使用，进一步解放 CPU

42. 中断的基本概念与分类

- **中断**：CPU 在执行程序过程中，因出现某些需要处理的事件，暂停当前程序转去执行相应服务程序，完成后返回原程序继续执行。
- 分类：
  - **内中断（异常）**：来自 CPU 内部，与当前指令相关，如**故障**（缺页、除数为 0 等，处理后可能重新执行引起故障的指令）、**自陷**（陷阱指令、系统调用、断点，返回下一条指令）、**终止**（硬件故障，无法恢复）；
  - **外中断（中断）**：来自 CPU 外部的设备请求，分为**可屏蔽中断**（INTR）与**不可屏蔽中断**（NMI，如掉电）。
- **响应中断的条件**：有中断请求；**CPU 允许中断（开中断，IF=1）**；且中断源未被屏蔽；**一条指令执行结束**时（缺页等异常除外，可在指令执行中响应）。

43. 中断处理过程

- **中断判优**：多个中断同时请求时由硬件排队器或软件查询决定**响应优先级**（一般：硬件故障 > 软件中断；不可屏蔽 > 可屏蔽；DMA 请求 > I/O 中断；高速设备 > 低速设备；输入 > 输出）。
- **中断隐指令**（由硬件自动完成，不是真正的指令）：
  1. **关中断**（防止保存断点期间被打断）；
  2. **保存断点**（PC 内容，通常压栈或存入特定单元；PSW 也会保存）；
  3. **引出中断服务程序**（通过**中断向量**/向量地址找到入口地址送入 PC）。
- **中断服务程序**（软件完成）：
  1. **保护现场**（保存通用寄存器等）；
  2. （多重中断时）**开中断**，允许更高级中断；
  3. **执行中断服务**（设备服务）；
  4. **关中断**；
  5. **恢复现场**；
  6. **开中断**；
  7. **中断返回**（恢复断点，返回原程序）。
- **向量中断**：每个中断源对应一个**中断向量**（服务程序入口地址），存放于**中断向量表**；中断源提供的**向量地址**用于查表得到入口地址。

44. 多重中断与中断屏蔽

- **多重中断（中断嵌套）**：在执行某中断服务程序时，允许响应更高优先级的中断。条件：服务程序中**提前开中断**，且新中断优先级更高。
- **中断屏蔽字**：每个中断源对应一个屏蔽字，某位为 **1 表示屏蔽**对应中断源；通常屏蔽自身及优先级比自己低的中断源。

- **响应优先级**由硬件排队电路决定，**不可改变**；**处理优先级**可通过**修改屏蔽字**动态改变。
- 常考：给定屏蔽字设置与中断请求时刻，画出 CPU 执行程序的轨迹。
- CPU 占用率计算：每次中断开销（含服务时间）× 每秒中断次数 / 1 秒。例：设备每 0.5ms 产生一次中断，每次处理 500 个时钟周期，CPU 主频 50MHz → 每秒中断 2000 次 × 500 = 10^6 周期 → 占 CPU 时间 10^6 / (50 × 10^6) = **2%**。

45. DMA 方式

- **DMA 控制器组成**：**主存地址计数器**（存放要交换数据的主存地址，每传一个字自动加 1）、**传送长度计数器（字计数器）**（记录剩余传送字数，减到 0 结束）、**数据缓冲寄存器**、DMA 请求触发器、**控制/状态逻辑**、**中断机构**（传送结束时向 CPU 发中断）、设备地址寄存器。
- **传送过程**：
  1. **预处理**（CPU 执行程序完成）：设置主存起始地址、传送字数、传送方向，启动设备；
  2. **数据传送**（DMA 控制器完成，不需要 CPU 执行程序）：每传送一个数据，地址加 1、字计数减 1；
  3. **后处理**：传送结束后 DMA **发出中断请求**，CPU 执行中断服务程序做校验、结束处理。
- **DMA 与主存交换数据的三种方式**：
  - **停止 CPU 访存**：DMA 传送期间 CPU 让出总线，控制简单，但 CPU 访存受阻；
  - **周期挪用（周期窃取）**：DMA 需要访存时**挪用一个或几个存取周期**；若与 CPU 同时访存，**DMA 优先**（I/O 设备数据不能丢失）；最常用；
  - **DMA 与 CPU 交替访存**：将一个 CPU 周期分为两半，分别供 DMA 和 CPU 访存，适合 CPU 周期大于主存周期的情况，硬件复杂。
- **DMA 与中断的区别**：
  - 中断通过 CPU **执行程序**传送数据，DMA 由**硬件**直接完成；
  - 中断需要**保护和恢复现场**，DMA 不需要（不改变 CPU 现场）；
  - **响应时机**：中断请求在**一条指令执行结束后**响应；DMA 请求在**每个机器周期（总线事务）结束后**即可响应；
  - **DMA 请求优先级高于中断请求**；
  - 中断适合低速、少量数据，并能处理异常事件；DMA 适合高速、成批数据传送，但传送结束后仍需中断进行后处理。
- 常考计算：如磁盘数据传输率 R、DMA 每次传送块大小 B、每次 DMA 预处理与后处理开销 c 个时钟周期，则每秒 DMA 次数 = R / B，CPU 占用率 = (R / B) × c / 主频。

46. 外部设备常考点

- **显示器**：分辨率、颜色深度（灰度级）、刷新频率；**显存容量 = 分辨率 × 颜色深度（位）/ 8**（单位字节）；**显存带宽 = 分辨率 × 颜色深度 × 帧频**（实际需考虑刷新和更新的双重访问）。例：1024×768、24 位真彩色 → 显存至少 1024 × 768 × 3B = 2.25MB。

- 键盘：扫描码与 ASCII 转换；打印机：针式、喷墨、激光。
- 外设与主机速度差异大，需要 I/O 接口缓冲与合适的控制方式。